

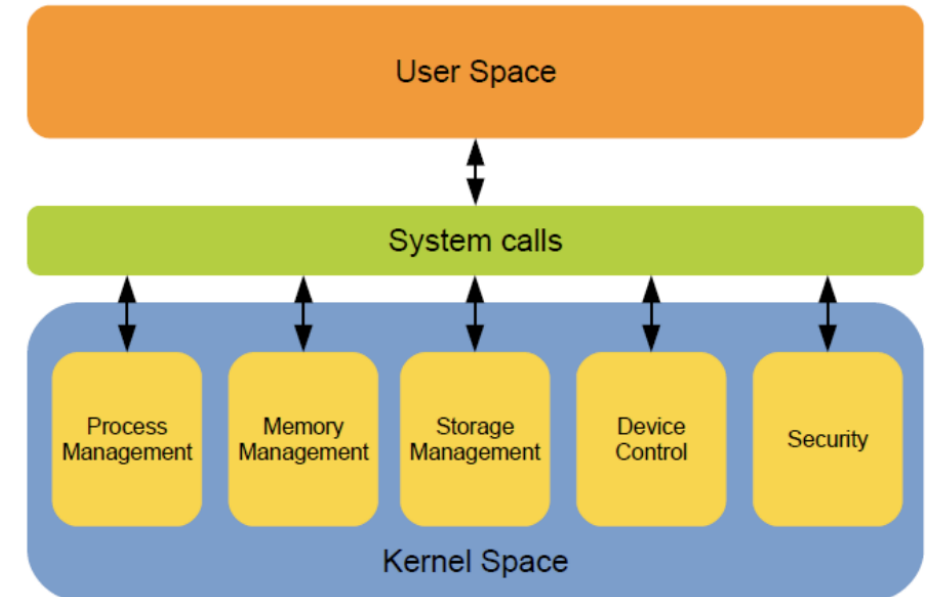


I/O Subsystem

Ivy Peng
Associate Professor
EECS, KTH

OS Organization

- Basic organization of an Operating System
 - Process management
 - Memory management
 - Device control and I/O subsystem
 - Storage management and File system
 - Protection and security



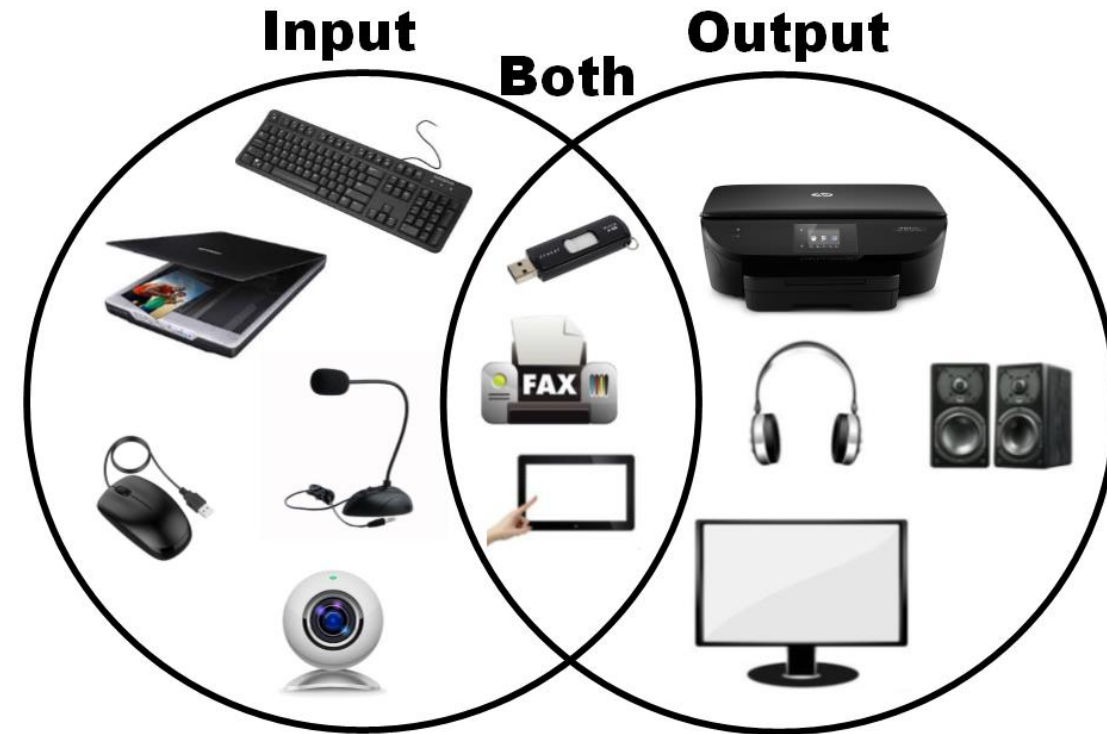


Outline

- **I/O Hardware**
- Host-Device Interaction
- Application I/O Interface
- Kernel I/O subsystem

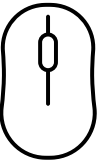
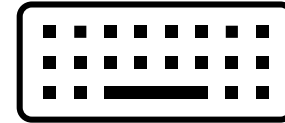
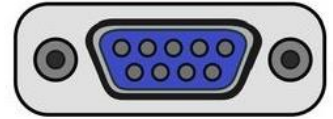
I/O Devices

- I/O devices are hardware components that either take data **input** or **output** data or both.
 - I/O stands for input and output
- There are an increasing variety of I/O devices:
 - Storage, e.g., disks, tapes
 - Transmission, e.g., network connections, Bluetooth
 - Human-interface, e.g., screen, keyboard, mouse, audio in and out
- OS needs to Manage and Control I/O operations and I/O devices
 - I/O subsystem of the kernel are responsible for I/O management



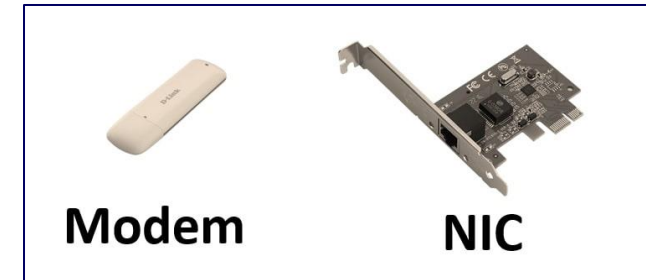
Character Devices

- The device transfers data in single bytes
- Example: keyboards, mouse, serial port



Network Devices

- Devices that interact and communicate with other devices over a network
- Linux, Unix, Windows include a **socket** interface for creating endpoints of communication and transferring data between endpoints
- the **socket** interface separates network protocol from network operations



Block Devices

- As introduced in Mass Storage, disk drives like HDD and NVM are common block devices
- The device transfers data in blocks: e.g., HDD in sectors, NVM in pages
- Common commands include **read()** and **write()** are used for transferring bytes from files





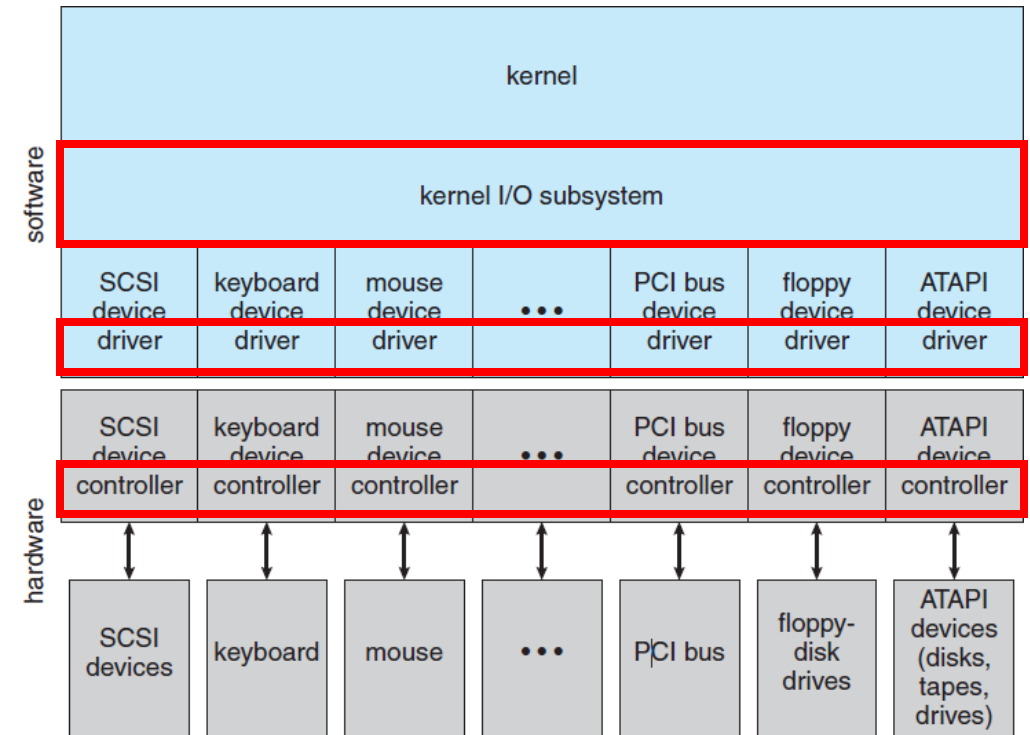
Outline

- I/O Hardware
- **Host-Device Interaction**
- Application I/O Interface
- Kernel I/O subsystem

An Overview of I/O Management

The hardware and software main components in I/O management

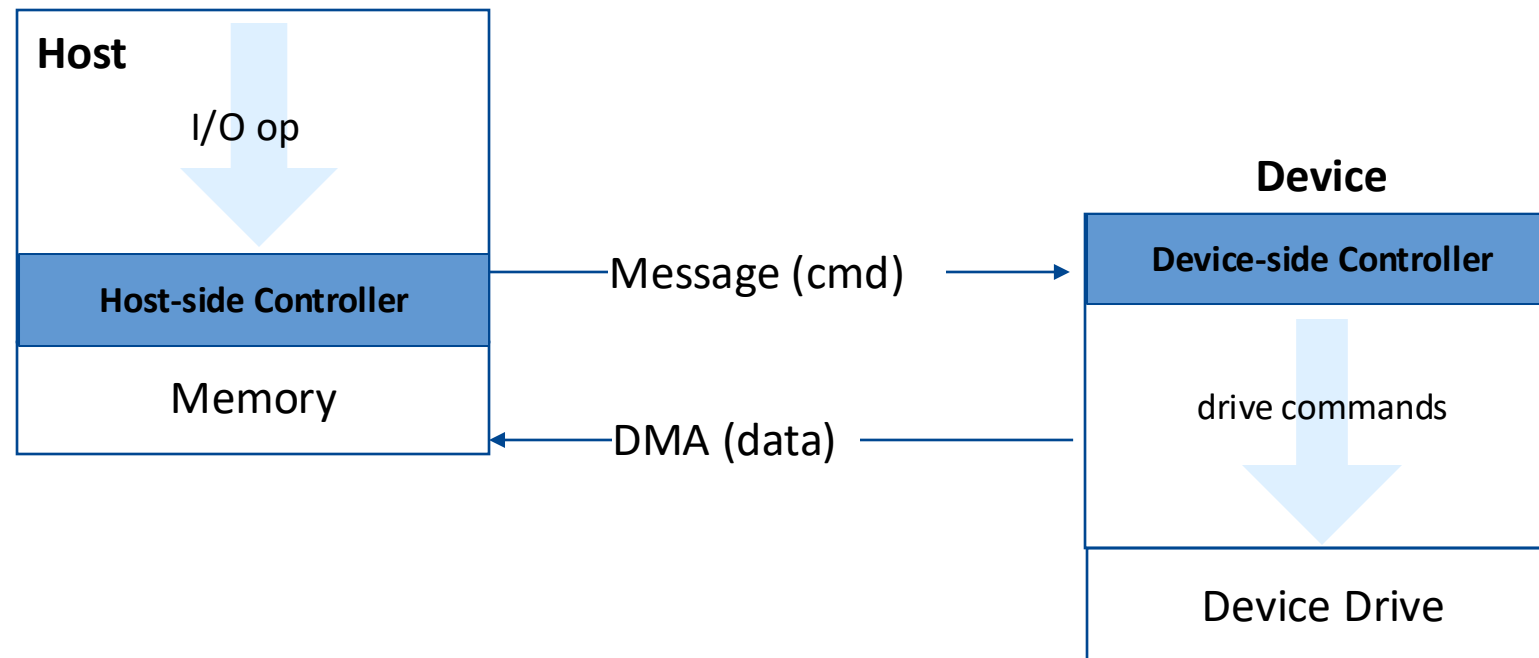
- Hardware: ports, busses, device controllers
- The kernel of an OS is structured to use device-driver modules
- Device drivers encapsulate device details and present a uniform interface to various devices



A Kernel I/O Structure

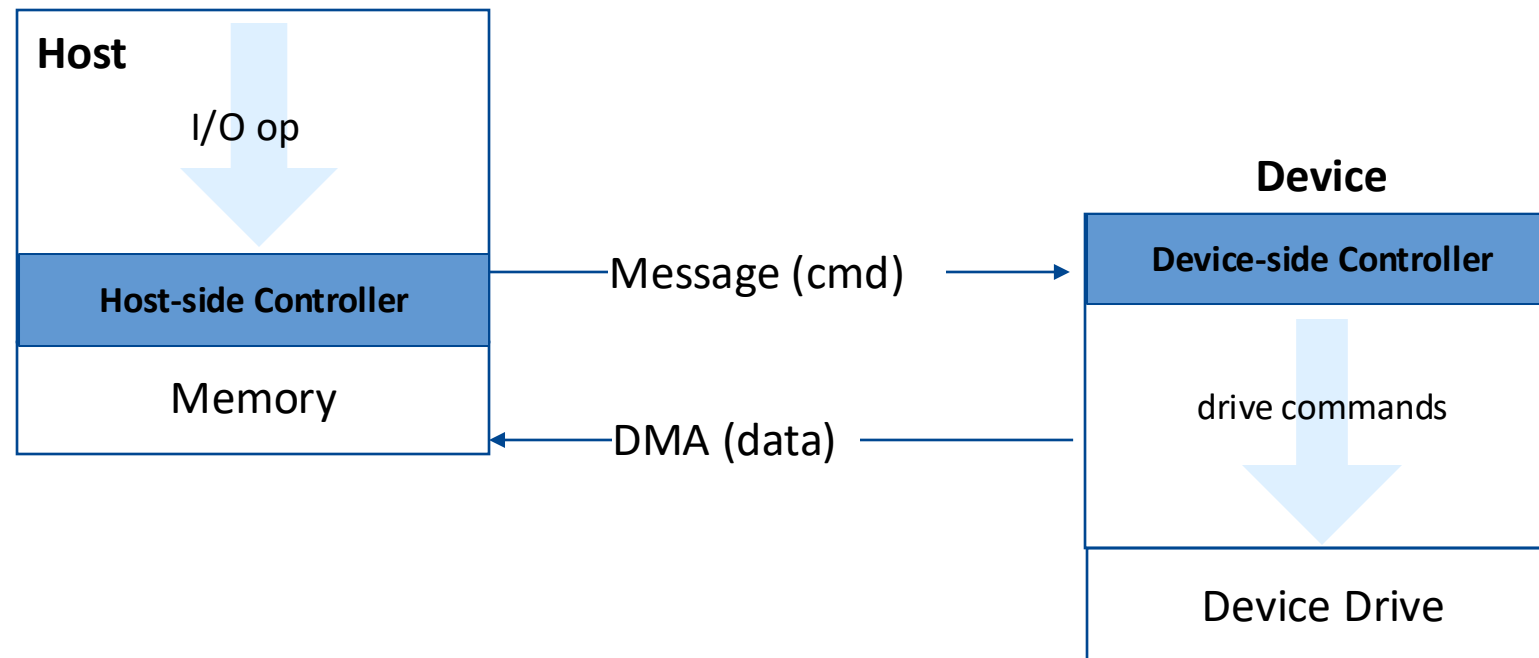
I/O Hardware Components

- **I/O Port:** connection point for device.
- **Bus:** a set of wires and protocols that specify how data can be transferred.
- **Controller:** special processors that operate a port, a bus, or a device. Run firmware and interact with device driver



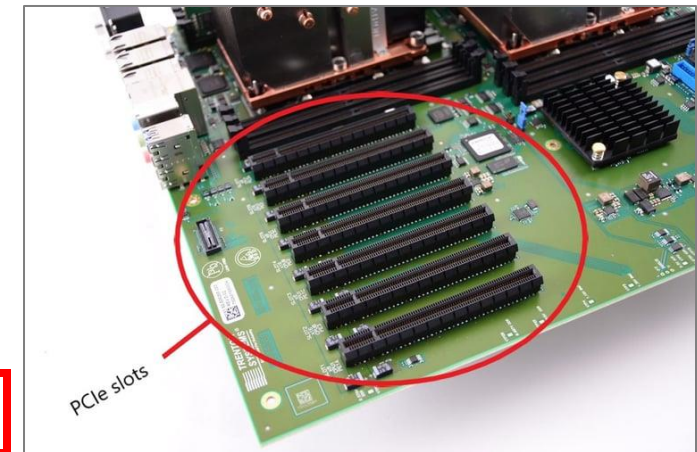
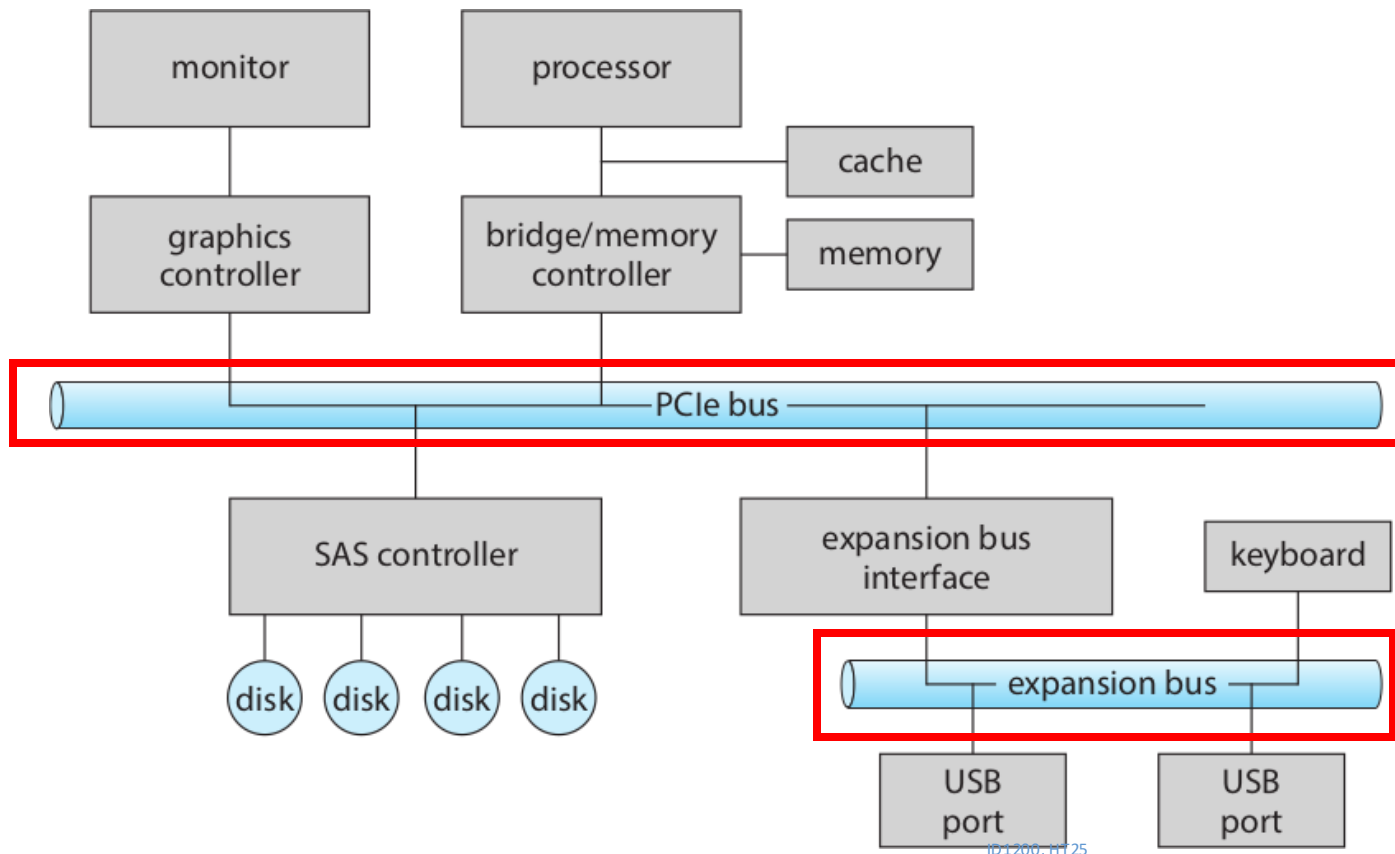
I/O Hardware Components: Port

- A I/O port typically has status, control, data-in, and data-out registers
- CPU communicates with device controllers (registers) in two ways:
 - Specific I/O port address. Use special I/O instructions to transfer data.
 - Memory-mapped I/O. Use memory. Use the same CPU instructions for memory access



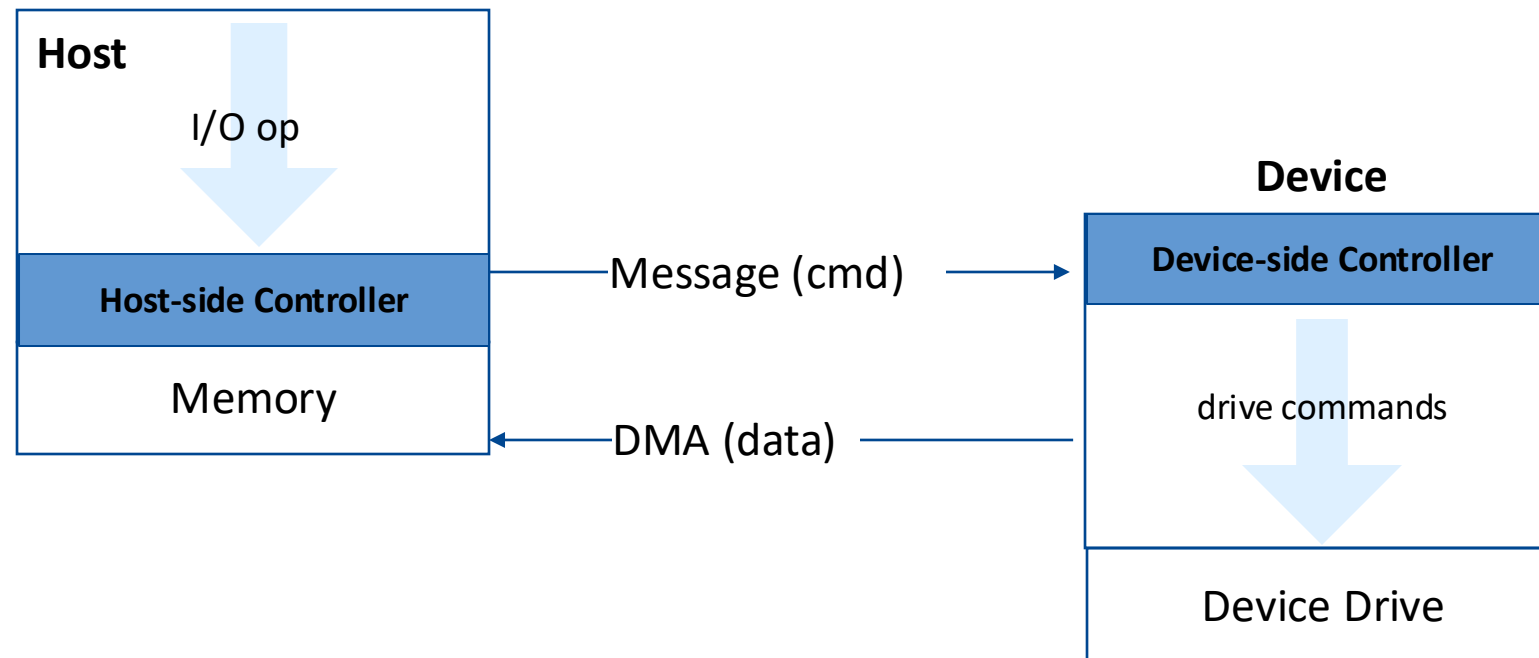
I/O Hardware Components: Bus

- PCIe bus: connects the processor's memory subsystem to fast devices.
 - PCIe: Peripheral Component Interconnect Express
- Expansion bus: connects relatively slow devices.



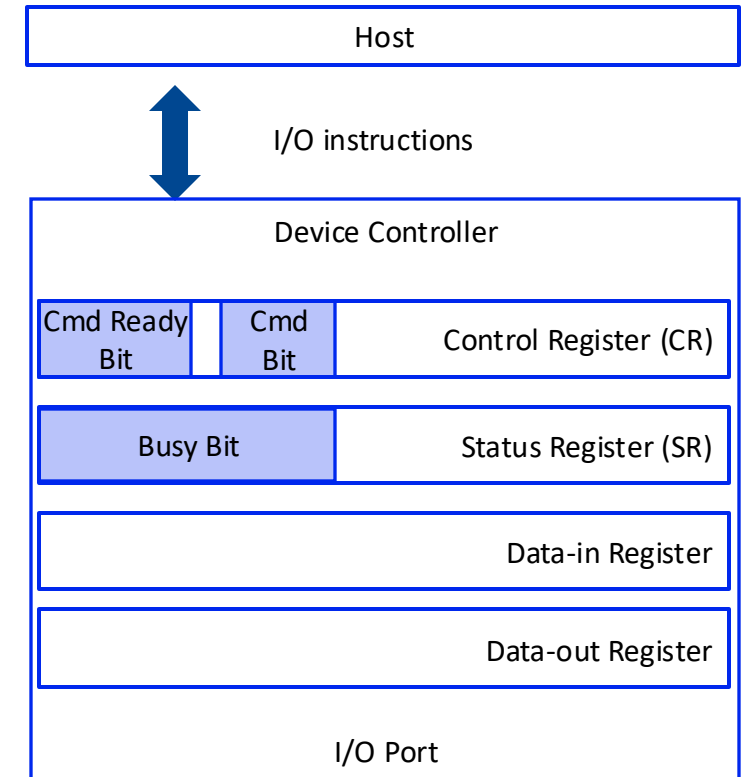
Host-Device Interaction

- Polling
- Interrupt
- DMA (Direct Memory Access)



Host-Device Interaction: Polling

- The processor communicates with the controller by reading and writing bit patterns in **registers**
- Host keeps reading Busy Bit in SR till it is 0. Then, it sets Command Bit:
 - If it is a Write, it also copy data into Data-out Register
- Host sets the Command Ready Bit.
- Next, the controller set Busy Bit to 1
 - The controller executes the command
 - When data transfer is done, Busy Bit, Command Ready Bit, are cleared
- Next, the controller set Busy Bit to 1 - Host **busy-waits** I/O from a device until it is ready
 - The controller executes the command
 - When data transfer is done, Busy Bit, Command Ready Bit, are cleared
- Host busy-waits the Ready Bit to be 0 from a device
 - Good for fast devices, but inefficient for slow devices



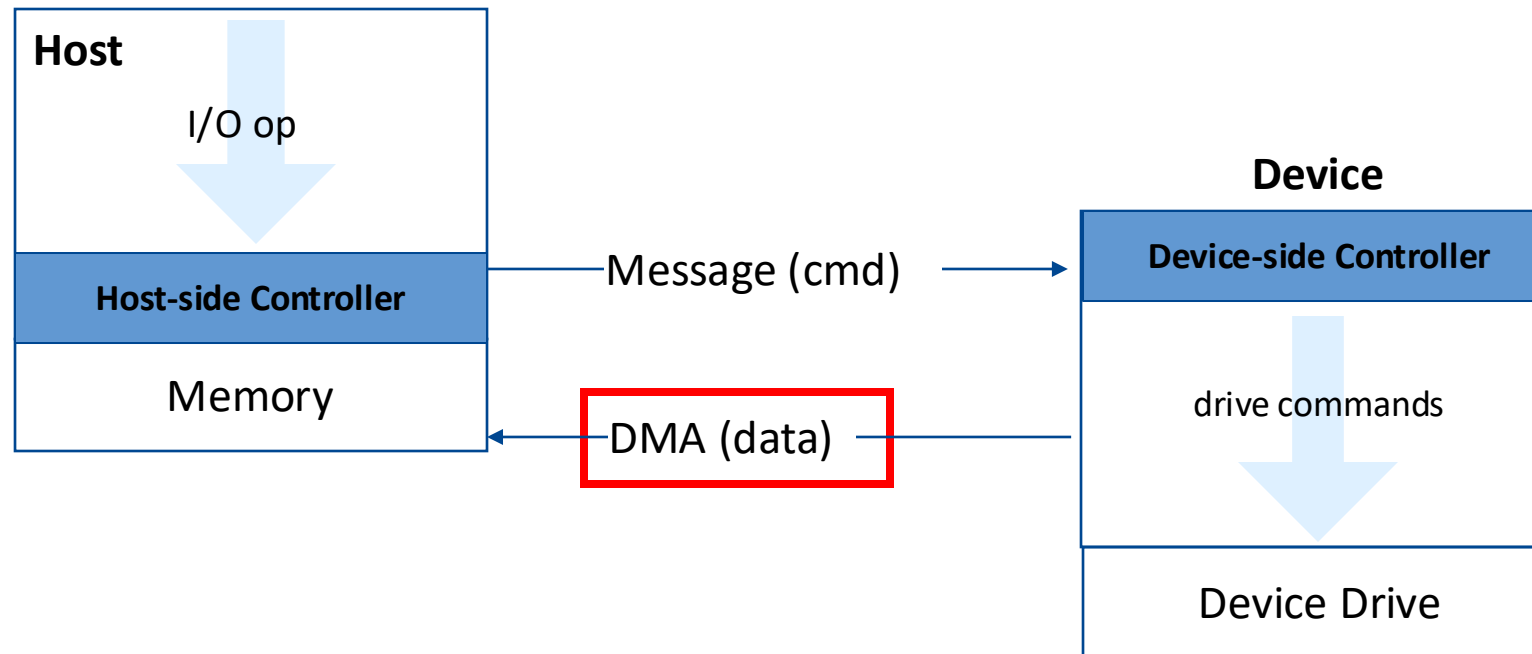
Host-Device Interaction: Interrupts

- **Interrupt** allows I/O devices to 'inform' the processor when I/O is ready
 - Host does not need to busy-wait
 - In parallel, the device executes I/O
 - CPU checks for interrupts between instructions
- When interrupted, the processor 'jumps' to an interrupt-handler routine at a fixed address in memory to process data
 - The interrupt mechanism accepts an **address**: a number that selects a specific interrupt-handling routine

vector number	description
0	divide error
1	debug exception
2	null interrupt
3	breakpoint
4	INTO-detected overflow
5	bound range exception
6	invalid opcode
7	device not available
8	double fault
9	coprocessor segment overrun (reserved)
10	invalid task state segment
11	segment not present
12	stack fault
13	general protection
14	page fault
15	(Intel reserved, do not use)
16	floating-point error
17	alignment check
18	machine check
19–31	(Intel reserved, do not use)
32–255	maskable interrupts

Host-Device Interaction: DMA (1/2)

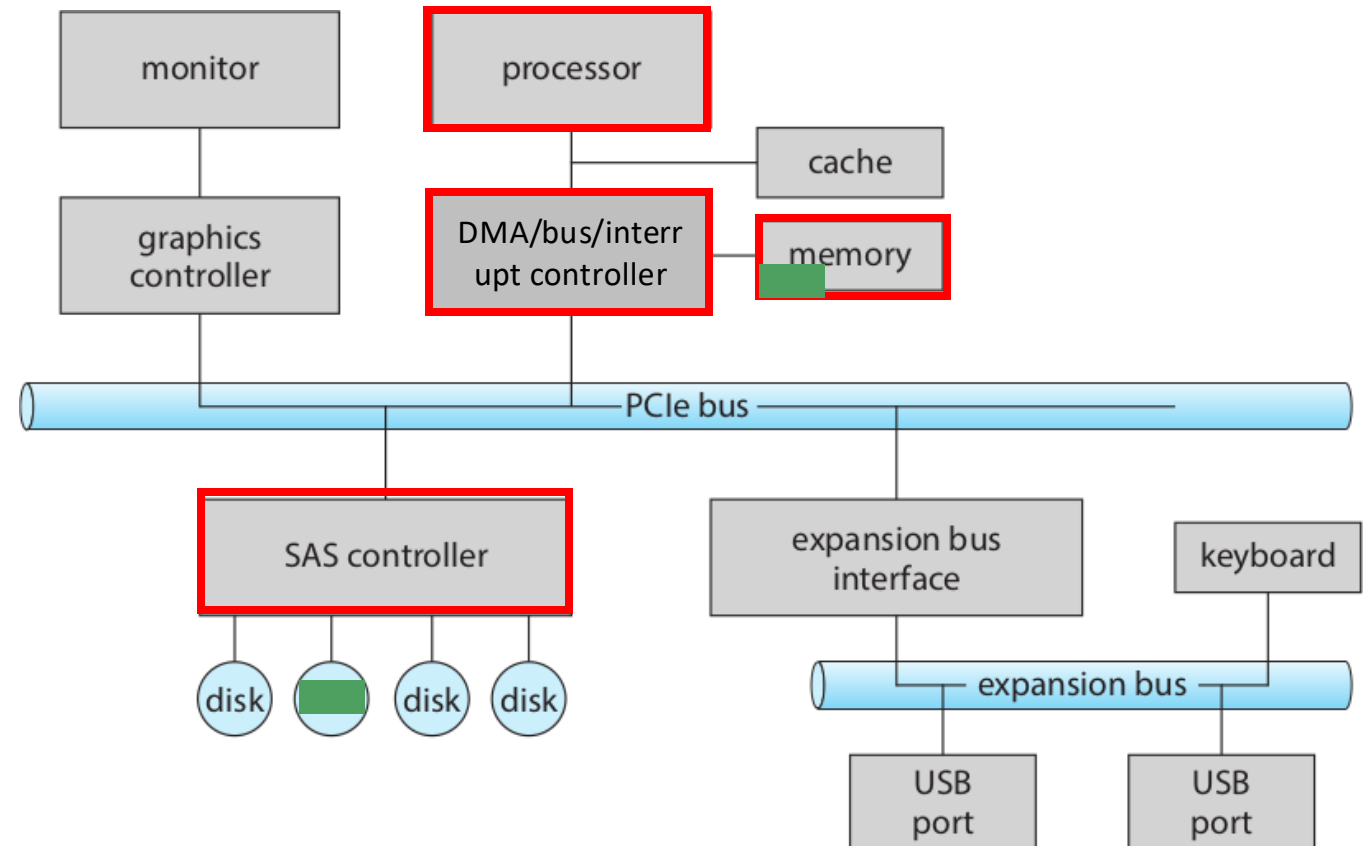
- Direct Memory Access (DMA) bypasses CPU to transfer data directly between I/O device and memory.



Host-Device Interaction: DMA (2/2)

A general workflow of DMA:

- Device driver is instructed to transfer data to address x
- Device driver instructs device controller
- Device controller initiates DMA transfer
- DMA controller transfers data
- DMA interrupts CPU to signal the completion of data transfer



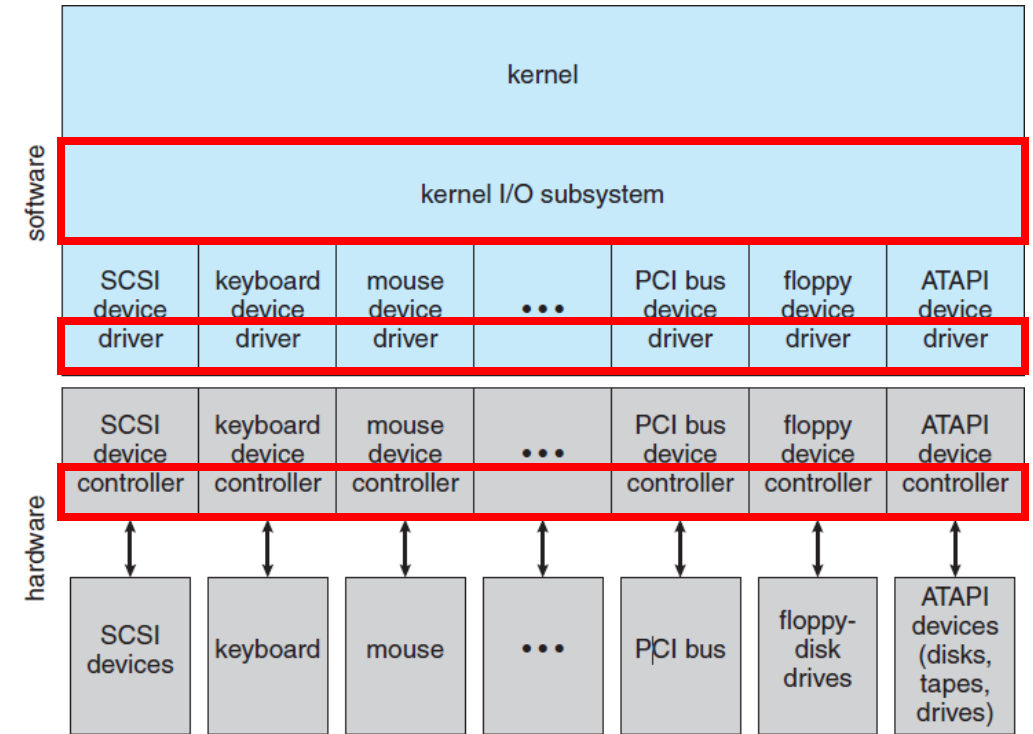


Outline

- I/O Hardware
- Host-Device Interaction
- **Application I/O Interface**
- Kernel I/O subsystem

Application I/O Interface

- **I/O system calls** encapsulate device behaviors in generic classes.
 - E.g., open, read, write, close
- Device-driver layer hides differences among I/O controllers from the kernel.



A Kernel I/O Structure

I/O Operations on Files

- Open

```
int open(const char *pathname, int flags, mode_t mode);
```

```
int fd = open("file.txt", O_RDWR | O_CREAT, 0666);
```

- Read

```
ssize_t read(int fd, void *buf, size_t count);
```

```
char buffer[1024];  
ssize_t bytes_read = read(fd, buffer, sizeof(buffer));
```

- Write

```
ssize_t write(int fd, const void *buf, size_t count);
```

```
const char *data = "My TEST Write Data !";  
ssize_t bytes_write = write(fd, data, strlen(data));
```

- Close

```
int close(int fd);
```

```
close(fd);
```

I/O Operations over Network

- In Linux, Unix, Windows, a socket is an endpoint for communicating data

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```

- Sockets on different machines are created to support communication over network
 - Different network protocols are supported
- Socket also supports inter-process communication on the same machine

```
#include <sys/socket.h>
#include <string.h>
#include <arpa/inet.h>
#include <stdio.h>

int main(int argc , char *argv[]) {
    int socket_desc;
    socket_desc = socket(AF_INET , SOCK_STREAM , 0);
    if(socket_desc<0){
        printf("socket failed \n");
        return 0;
    }

    struct sockaddr_in server;
    server.sin_addr.s_addr = inet_addr("172.217.21.164");
    server.sin_family = AF_INET;
    server.sin_port = htons(80);

    if(connect(socket_desc , (struct sockaddr *)&server , sizeof(server))<0){
        printf("connect failed \n");
        return 0;
    }
    printf("Connected\n");

    char *msg = "hello!";
    if(send(socket_desc ,msg, strlen(msg) , 0)<0)
        printf("send failed\n");
    printf("send\n");

    return 0;
}
```

Blocking and Nonblocking I/O

- **Blocking:** When a process performs a blocking I/O operation, it will block until the operation is complete.
 - E.g., `read()` and `write()` are mostly used as blocking I/O.
- **Nonblocking:** the process does not wait for the I/O operation to complete, but it needs to test whether any data become available (and transfer them)
 - E.g., `poll()`, **`select()`**, or `epoll()` are commonly used to manage multiple I/O operations

Pseudo code for blocking reads from N sockets

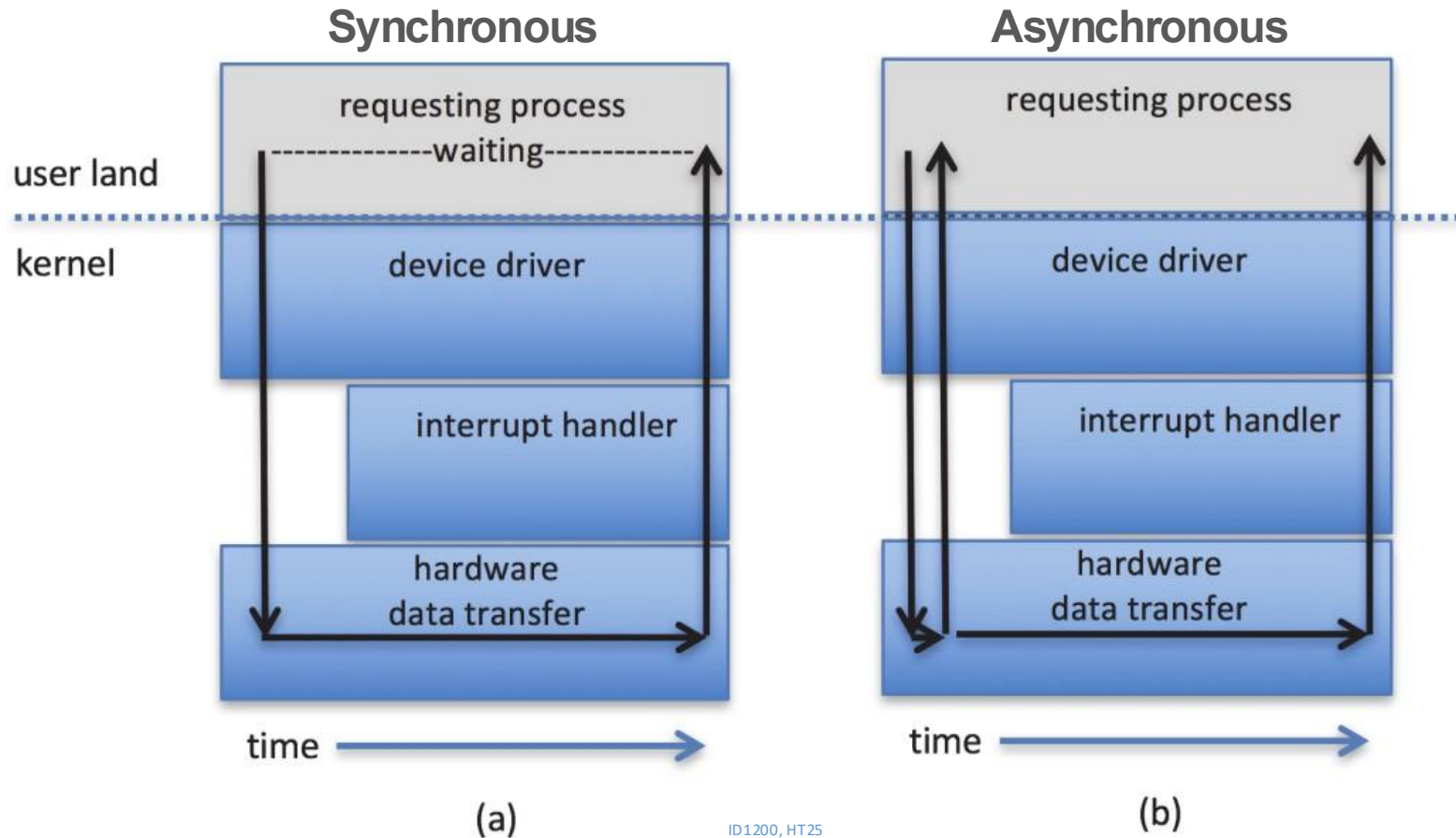
```
processed = 0
for (;processed<num_sockets; processed++) {
    read(socket, buffer);
}
```

Pseudo code for Nonblocking reads from N sockets

```
processed = 0
while (processed<num_sockets) {
    sockets = select();
    for (socket in sockets) {
        read(socket, buffer);
        processed++;
    }
}
```

Asynchronous I/O

- Process runs while I/O executes. I/O subsystem **signals** process when I/O completed.
- Difference from nonblocking I/O? the OS completes the I/O in the background



I/O Activities in Linux

```
~$ iostat
```

```
avg-cpu:  %user   %nice %system %iowait  %steal   %idle
            0,14    0,01    0,10    0,02    0,00   99,72

Device            tps    kB_read/s    kB_wrtn/s    kB_dscd/s    kB_read    kB_wrtn    kB_dscd
vda                4,69         11,26         387,77         113,46    5732433    197485360    57781292
vda1               0,00          0,03          0,00          3,05      16728          0      1555824
vda2               4,68         11,18         387,77         110,40    5694937    197485360    56225468
```

- %iowait: CPUs idling while there is an outstanding disk I/O request.



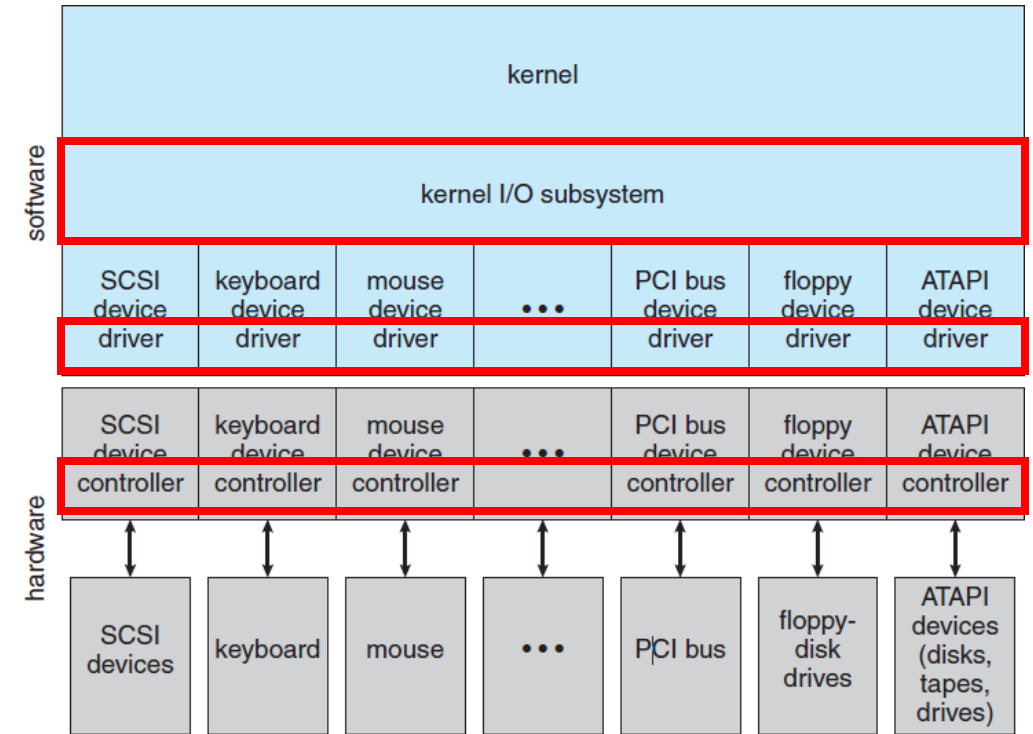
Outline

- I/O Hardware
- Host-Device Interaction
- Application I/O Interface
- **Kernel I/O subsystem**

Kernel I/O Subsystem

Kernels provide the following services related to I/O:

- Scheduling
- Buffering
- Caching
- Error handling



A Kernel I/O Structure

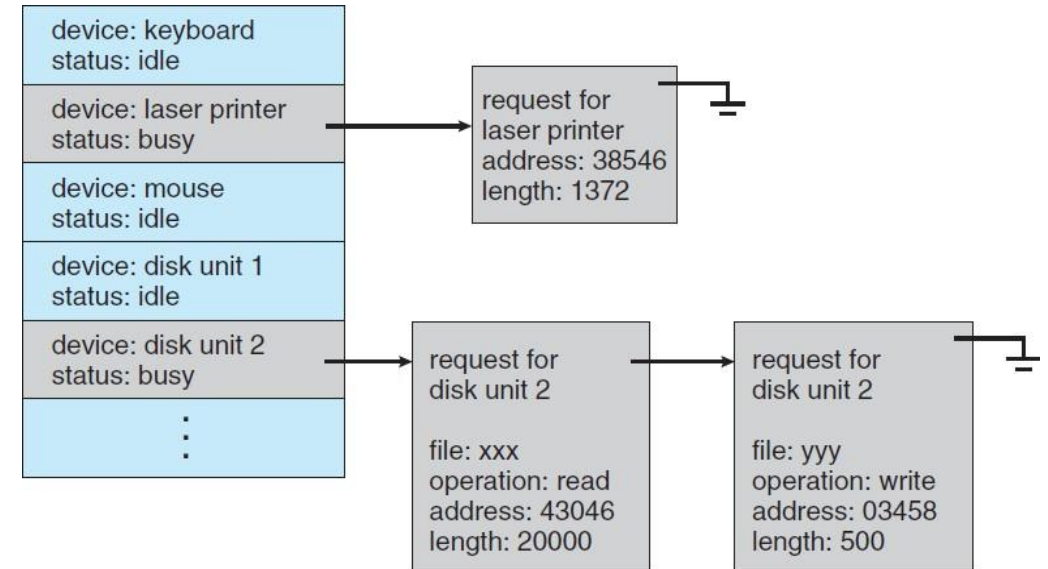


I/O Scheduling

- Determine a good order in which to execute I/O requests.
- Some I/O request ordering via per-device queue.
- Different metrics for different operating systems:
 - Timeliness
 - Fairness
 - ...

Scheduling for Asynchronous I/O

- The kernel must keep track of many I/O requests at the same time.
- The OS attaches the wait queue to a **device-status table**.
- The table contains an entry for each I/O device.
- A wait queue is used to store pending to a device if it is busy
 - If the device is busy with a request, the type of request and other parameters will be stored in the table entry for that device.





Buffering

- Buffering: stores data in memory while transferring between devices.
 - help cope with device speed mismatch.
 - help cope with device transfer size mismatch, e.g., fragmentation and reassembly of messages.
 - help maintain copy semantics.

Caching

- The faster device holds a copy of data:
 - the Linux system keeps a **page cache** in memory
- Is key to performance
 - Bring the performance to slow disks

```
$ free -b
```

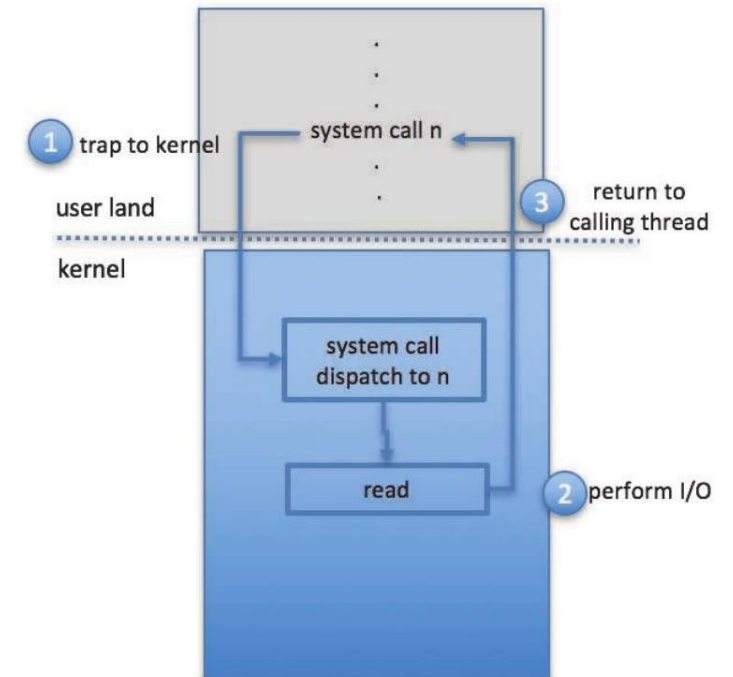
```
Mem:          total        used        free      shared  buff/cache   available
Swap:      4399820800       11079680  4388741120
```

Error Handling

- OS can recover from disk read, device unavailable, and transient write failures.
 - Retry a read or write.
 - Track error frequencies, stop using device with increasing frequency of retry-able errors.
- Most return an error number when I/O request fails.
 - In UNIX, an integer variable errno is used to return an error code
- System error logs hold problem reports.

I/O Protection

- A user process may accidentally or purposefully attempt to disrupt normal operation via illegal I/O instructions.
- OS needs to prevent users from illegal I/O
- Thus, users cannot issue I/O instructions directly. I/O must be performed via system calls.
- Also, memory-mapped and I/O port memory locations must be protected from user access





Summary

- I/O Hardware
- Host-Device Interaction
- Application I/O Interface
- Kernel I/O subsystem